

6 Perform case study of regression testing using JUnit Framework.

Regression testing is a type of software testing performed to ensure that recent code changes do not negatively affect the existing functionality of an application. Whenever developers fix bugs, add features or refactor code regression testing verifies that previously working modules still function correctly.

JUnit is a widely used testing framework for java that supports automated unit testing and regression testing. It provides annotations, assertions and test runners that make testing efficient and repeatable.

Problem Statement:

Consider a bank account management system developed in Java. The system supports the following operations.

- Deposit money
- Withdraw money
- Check balance

Initially the application worked correctly.

Later developer introduced a new feature.

A transaction fee is applied for withdrawals above 5000.

After modifying the withdrawal method

regression testing is required to ensure that

existing deposit functionality still works. Balance calculation remains correct. Previous withdrawal logic is not broken ~~any~~.

System Implementation.

```
Public class BankAccount {  
    Private double balance;  
    public BankAccount (double balance) {  
        this.balance = balance;  
    }  
    public void withdraw (double amount) {  
        balance = balance + amount;  
    }  
    public void withdraw (double amount) {  
        if (amount <= balance) {  
            balance = balance - amount;  
        }  
    }  
    public double get Balance () {  
        return balance;  
    }  
}
```

Modified code After Feature-addition.

```
Public void withdraw (double amount) {  
    if (amount <= balance) {  
        if (amount > 5000) {  
            balance = balance - (amount + 50);  
        }  
        else {  
            balance = balance - amount;  
        }  
    }  
}
```


This change may affect previous functionality, so regression testing is necessary.

Junit allows developers to write automated test cases that can be executed repeatedly after every change.

```
import static org.junit.jupiter.api.Assertions.*;
import org.junit.jupiter.api.Test;
public class BankAccountTest {
```

```
    @Test
```

```
    void testDeposit() {
```

```
        BankAccount acc = new BankAccount(1000);
```

```
        acc.deposit(500);
```

```
        assertEquals(1500, acc.getBalance());
```

```
    }
```

```
    @Test
```

```
    void testWithdrawWithoutFee() {
```

```
        BankAccount acc = new BankAccount(6000);
```

```
        acc.withdraw(3000);
```

```
        assertEquals(3000, acc.getBalance());
```

```
    }
```

```
    @Test
```

```
    void testInsufficientBalance() {
```

```
        BankAccount acc = new BankAccount(2000);
```

```
        acc.withdraw(5000);
```

```
        assertEquals(2000, acc.getBalance());
```

```
    }
```

```
}
```


Regression testing process.

- i) Developer modify code
- ii) Existing JUnit test codes are excluded.
- iii) If any test fails regression bug tested.
- iv) Developer fixes issue.
- v) Tests are reexecuted until all pass.

Test results:

Test case	Expected Result	Actual Result	Status
Deposit test	1500	1500	Pass
Withdraw without fee	3000	3000	Pass
Withdraw with fee	3050	3050	Pass
Insufficient Balance	2000	2000	Pass

All regression test passed confirming that new changes did not break existing functionality.

This test study demonstrates how regression testing can be effectively performed using the JUnit framework. Automated regression testing ensures software reliability by verifying that modifications don't introduce.